

Trabajo Práctico de Prueba 1: Entorno de Trabajo Profesional y Control de Versiones

Alumno: Sussini Patricio

Materia: Metodología de Sistemas I

Sección A: Análisis y Justificación del Entorno de Desarrollo (IDE)

1. Adopción de Visual Studio Code (VS Code):

- **Justificación:** La transición de editores de texto convencionales hacia un Entorno de Desarrollo Integrado (IDE) es un imperativo en el desarrollo de software moderno. La elección de VS Code radica en su arquitectura unificada, la cual consolida las fases de edición, ejecución en terminal, depuración y control de versiones en una única interfaz. Esto minimiza el cambio de contexto (context switching) y optimiza drásticamente el rendimiento operativo del desarrollador.

2. Impacto de las Herramientas Integradas en la Calidad del Código:

- **IntelliSense y Linting:** El autocompletado trasciende la mera asistencia tipográfica, operando como un analizador estático en tiempo real. En conjunción con herramientas de linting, se establece una primera barrera defensiva que neutraliza vulnerabilidades sintácticas y "code smells" de forma preventiva, previo a cualquier intento de ejecución.
- **El Depurador (Debugger):** Constituye el núcleo del diagnóstico de software. Suprime la práctica arcaica e ineficiente de la depuración por impresión (ej. `console.log`), ofreciendo un entorno controlado mediante Breakpoints. Esta funcionalidad facilita la suspensión del hilo de ejecución, la evaluación de expresiones y el monitoreo del estado de las variables en tiempo real, permitiendo una resolución de incidencias sistemática y precisa.

Sección B: Estrategia de Control de Versiones

1. Diferenciación Arquitectónica (Git vs. GitHub):

- **Explicación:** Resulta imperativo discernir entre la tecnología subyacente y el servicio de alojamiento. Git opera como el motor de control de versiones distribuido a nivel local, garantizando la trazabilidad criptográfica y matemática de las mutaciones del código. Por su parte, GitHub funciona como la infraestructura en la nube que centraliza dichos repositorios, proveyendo la arquitectura necesaria para la integración continua, las auditorías de código mediante Pull Requests y la redundancia de datos a nivel corporativo.

2. El Flujo de Trabajo Seguro (Las 3 Áreas de Git):

- **Flujo Operativo:** Para asegurar la atomicidad y coherencia de las confirmaciones (commits), el ciclo de vida del código se segmenta en tres fases críticas:
 1. **Working Directory (Directorio de Trabajo):** El entorno volátil donde se efectúan las modificaciones del código fuente en tiempo real.
 2. **Staging Area (Área de Preparación):** La zona de indexación (git add). Actúa como un filtro selectivo que permite aislar y agrupar lógicamente los archivos que compondrán el próximo registro histórico, evitando mezclar funcionalidades inconexas.
 3. **Local Repository (Repositorio Local):** La base de datos permanente (git commit) donde se consolida la instantánea del proyecto, exigiendo siempre un mensaje descriptivo que justifique semánticamente el "por qué" de la alteración introducida.

Sección C: Gestión de Colaboración

1. Estrategia de Ramas (Branches):

- **Justificación:** Operar directamente sobre la rama principal (main/master) constituye una vulnerabilidad arquitectónica inaceptable en entornos colaborativos, dado que esta debe reflejar invariablemente un estado estable y listo para producción. La implementación de un modelo basado en *feature branches* (ramas de características) confiere un entorno aislado (sandbox) donde cada desarrollador puede iterar, construir y fallar de manera segura sin corromper la rama troncal. Posteriormente, los comandos push y pull orquestan la sincronización y convergencia ordenada del trabajo con el resto del equipo.